

Load Balancers

Task-1. In the AWS Management Console, launch two EC2 instances running a simple web server (use Amazon Linux 2 and install httpd or nginx), and verify both are accessible via their public IPs.

We will create:

- **EC2-Server-1**
- **EC2-Server-2**

Both will run Apache (httpd).

Step 1: Launch the first EC2 instance

1. Log in to the **AWS Management Console**.
2. Search for **EC2** and open it.
3. Click **Instances** → **Launch instances**.
4. Enter:

Name:

Server-1

5. Under **Application and OS Images**, select **Amazon Linux 2 AMI**.
6. Select:

Instance type: t3.micro

if it is available/eligible for your account.

7. Select or create a **Key pair**.
8. Under **Network settings**, select your VPC/subnet.
9. Make sure the Security Group allows:

Type Port Source

SSH 22 My IP

HTTP 80 Anywhere IPv4

10. Click **Launch instance**.

Step 2: Launch the second EC2 instance

Repeat the same process.

Use:

Name: Server-2

Use the same VPC and preferably the **same security group**.

After both instances are running, your EC2 page should look approximately like:

Instance State	Private IP	Public IP
Server-1 Running	x.x.x.x	xxx.xxx.xxx.xxx
Server-2 Running	x.x.x.x	xxx.xxx.xxx.xxx

Step 3: Connect to Server-1

Select **Server-1** → **Connect** → **SSH client**.

Copy the SSH command provided by AWS. It will look similar to:

```
ssh -i "my-key.pem" ec2-user@YOUR_PUBLIC_IP
```

If necessary, on your local computer:

```
chmod 400 webserverkey.pem
```

Step 4: Install Apache on WebServer-1

After connecting:

```
sudo yum update -y
```

```
Install Apache: - sudo yum install httpd -y
```

```
Start Apache:- sudo systemctl start httpd
```

```
Enable it at startup:- sudo systemctl enable httpd
```

```
Create a webpage:- sudo vi /var/www/html/index.html
```

Put this in the file:

```
<!DOCTYPE html>

<html>

<head>

  <title>Web Server 1</title>

</head>

<body>

  <h1>Hello from Web Server 1</h1>

  <p>This page is running on EC2 Instance 1.</p>

</body>

</html>
```

Save with:

:wq

Step 5: Install Apache on WebServer-2

Connect to WebServer-2 using SSH and run:

```
sudo yum update -y
```

```
sudo yum install httpd -y
```

```
sudo systemctl start httpd
```

```
sudo systemctl enable httpd
```

Create the webpage:

```
sudo vi /var/www/html/index.html
```

Enter:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Web Server 2</title>
</head>
<body>
  <h1>Hello from Web Server 2</h1>
  <p>This page is running on EC2 Instance 2.</p>
</body>
</html>
```

Save it.

Step 6: Test both servers

Copy the **Public IPv4 address** of WebServer-1 and open:

`http://PUBLIC-IP-1`

You should see:

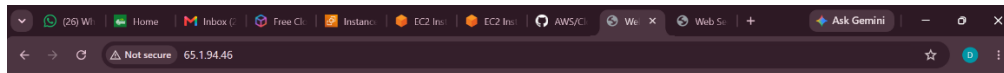
Hello from Web Server 1

Then open:

`http://PUBLIC-IP-2`

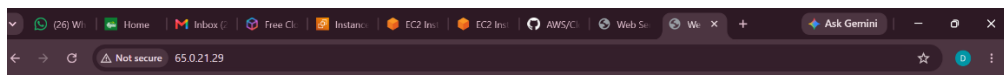
You should see:

Hello from Web Server 2



Hello from Web Server 1

This page is running on EC2 Instance 1.



Hello from Web Server 2

This page is running on EC2 Instance 2.



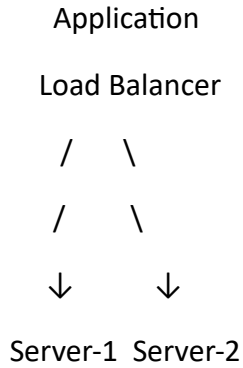
Task-2.Set up an Application Load Balancer (ALB) in AWS that distributes incoming HTTP traffic across your two EC2 instances, and test by accessing the ALB's DNS name in your browser to see traffic served from both servers.

The architecture will be:

Internet

|

↓



Step 1: Create a Target Group

1. Open **EC2**.
2. In the left menu, select **Target Groups**.
3. Click **Create target group**.
4. Select:

Target type: Instances

5. Enter:

Target group name: WebServers-TG

6. Protocol: HTTP
7. Port: 80
8. Select your VPC: default vpc or whichever VPC contains your EC2 instances.
9. For the health check, initially use:

Protocol: HTTP

Path: /

10. Click **Next**.
11. Select both: Server-1 , Server-2
12. Click **Include as pending below**.
13. Click **Create target group**.

Step 2: Create the Application Load Balancer

1. Go to **EC2 → Load Balancers**.
2. Click **Create Load Balancer**.
3. Select: **Application Load Balancer**
4. Click **Create**.

Enter:

Load balancer name: my-ALB

5. Scheme: Internet-facing
6. IP address type: IPv4

Step 3: Select your VPC and subnets

Select your VPC.

For an internet-facing ALB, select **at least two subnets in different Availability Zones** if available.

For example:

ap-south-1a

ap-south-1b

Step 4: Configure the Security Group

Create/select a Security Group that allows:

Type Port Source

HTTP 80 Anywhere IPv4

For better security, the **EC2 security group** should ideally allow HTTP traffic from the **ALB security group**, rather than from the entire internet.

Step 5: Configure Listener

Under **Listeners and routing**, configure:

Protocol: HTTP

Port: 80

Default action: Forward to WebServers-TG

Click **Create load balancer**.

Step 6: Test the ALB

After the ALB is created:

1. Go to **EC2 → Load Balancers**.
2. Select Zomato-ALB.
3. Copy its **DNS name**.

It will look something like:

Zomato-ALB-123456.ap-south-1.elb.amazonaws.com

Open:

http://YOUR-ALB-DNS-NAME

You should see either:

Hello from Web Server 1

or:

Hello from Web Server 2

Refresh/send multiple requests. The ALB distributes requests between the healthy instances.

aws [Search] [Alt+S] Ask Amazon Q Asia Pacific (Mumbai) Mehta Darshit (85627778179) Mehta Darshit

EC2 > Load balancers

Load balancers (1/1) What's new?

Elastic Load Balancing scales your load balancer capacity automatically in response to changes in incoming traffic.

Filter load balancers

Name	State	Type	Scheme	IP address type	VPC ID	Avail
server-alb	Active	application	Internet-facing	IPv4	vpc-0339abc6810a7778c	2 Av

Drag or select to resize

Load balancer: server-alb

< Details Listeners and rules Network mapping Resource map Security Monitoring Integrations Attribution >

Details

Load balancer type	Status	VPC	Load balancer IP address type
Application	Active	vpc-0339abc6810a7778c	IPv4

Scheme: Internet-facing Hosted zone: Availability Zones: Data created:

CloudShell Agent Toolkit for AWS Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences 30°C Mostly cloudy 12:00 PM 07-Sep-26

ap-south-1.console.aws.amazon.com/ec2/home?region=ap-south-1#LoadBalancers:

aws [Search] [Alt+S] Ask Amazon Q Asia Pacific (Mumbai) Mehta Darshit (85627778179) Mehta Darshit

EC2 > Load balancers

Load balancers (1/1) What's new?

Elastic Load Balancing scales your load balancer capacity automatically in response to changes in incoming traffic.

Filter load balancers

Name	State	Type	Scheme	IP address type	VPC ID	Availability Zones	Security groups
server-alb	Active	application	Internet-facing	IPv4	vpc-0339abc6810a7778c	2 Availability Zones	sg-02f7c170ac5a

Load balancer: server-alb

Details

Load balancer type	Status	VPC	Load balancer IP address type
Application	Active	vpc-0339abc6810a7778c	IPv4

Scheme: Internet-facing Hosted zone: ZP97RAFLXTNZK Availability Zones: subnet-2593566c1680664d5 ap-south-1b (aps1-a25) subnet-07afb759db7c4022d8 ap-south-1a (aps1-a21) Date created: September 7, 2026, 11:54 (UTC+05:30)

Load balancer ARN: arnaws:elasticloadbalancing:ap-south-1:85627778179:loadbalancer/app/server-alb/5db2886e9104171f DNS name info: server-alb-1944481621.ap-south-1.elb.amazonaws.com (A Record)

CloudShell Agent Toolkit for AWS Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences 30°C Mostly cloudy 12:00 PM 07-Sep-26

aws

Search

[Alt+5] Ask Amazon Q

Asia Pacific (Mumbai)

Mehta Darshit (856727776179)

EC2 > Target groups

reserved instances
Dedicated Hosts
Capacity Reservations
Capacity Manager [New](#)

▼ Images
AMIs
AMI Catalog

▼ Elastic Block Store
Volumes
Snapshots
Lifecycle Manager

▼ Network & Security
Security Groups
Elastic IPs
Placement Groups
Key Pairs
Network Interfaces

▼ Load Balancing
Load Balancers
[Target Groups](#)
Trust Stores

Target groups (1/1) Info | What's new?

Filter target groups

Actions

Create target group

☒	Name	ARN	Port	Protocol	Target type	Load balancer
☒	webserver-tg	arn:aws:elasticloadbalancing...	80	HTTP	Instance	server-alb

Target group: webserver-tg

Filter targets

☐	Instance ID	Name	Port	Zone	Health status	Health status details	Admini..
☐	i-0983b11bbb6ad0711	server-2	80	ap-south-1b (a...	Healthy	-	-
☐	i-0c733f140da3efd62	server-1	80	ap-south-1b (a...	Healthy	-	-

CloudShell Agent Toolkit for AWS Feedback Console Mobile App

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

30°C Mostly cloudy

Search

ENG IN 12:03 PM 07-Sep-26

(26) Home Inbo Free Secu Creat EC2 EC2 AWS Web Web

Ask Gemini

server-alb-1944481621.ap-south-1.elb.amazonaws.com

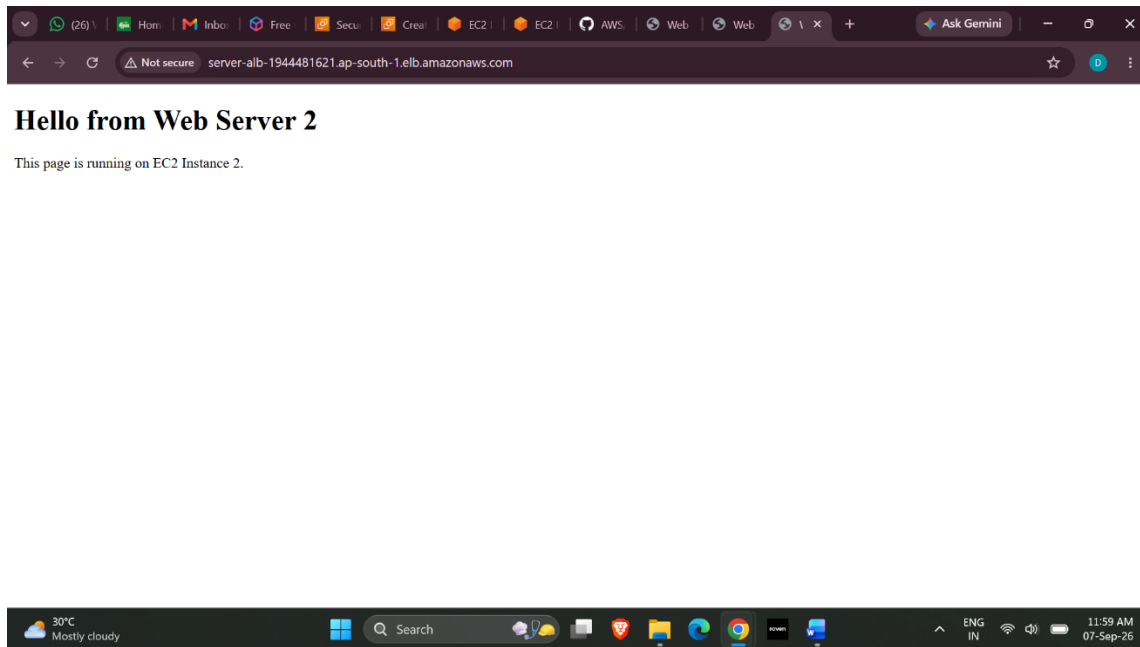
Hello from Web Server 1

This page is running on EC2 Instance 1.

30°C Mostly cloudy

Search

ENG IN 11:59 AM 07-Sep-26



Task-3.Change the health check settings of your ALB to use a custom path (e.g., /health) and verify that the load balancer only routes traffic to healthy instances.Create a /health endpoint on your web server that returns a 200 OK response.

Step 1: Create /health on WebServer-1

SSH into WebServer-1:

```
ssh -i "webserver.pem" ec2-user@PUBLIC-IP-1
```

Create the health page:

```
echo "OK - Web Server 1 is healthy" | sudo tee /var/www/html/health
```

Test:

```
curl http://localhost/health
```

You should get:

OK - Web Server 1 is healthy

Step 2: Create /health on WebServer-2

SSH into WebServer-2 and run:

```
echo "OK - Web Server 2 is healthy" | sudo tee /var/www/html/health
```

Test:

```
curl http://localhost/health
```

You should get:

```
OK - Web Server 2 is healthy
```

Step 3: Change the ALB health check

Go to:

EC2 → Target Groups → WebServers-TG → Health checks

Click **Edit**.

Change:

Health check protocol: HTTP

Health check path: /health

Health check port: Traffic port

Keep the success code as:

200

Save the changes.

Step 4: Check target health

Go to:

Target Groups → WebServers-TG → Targets

You should see:

Target Health

Server-1 Healthy

Server-2 Healthy

The ALB now requests:

/health

from each server.

If the server responds with HTTP **200 OK**, it is considered healthy.

```
aws
[Alt+S]
Installing : httpd filesystem-2.4.68-1.amzn2023.0.1.noarch 8/13
Installing : httpd-core-2.4.68-1.amzn2023.0.1.x86_64 9/13
Installing : mod_lua-2.4.68-1.amzn2023.0.1.x86_64 10/13
Installing : generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch 11/13
Installing : httpd-2.4.68-1.amzn2023.0.1.x86_64 12/13
Running scriptlet: httpd-2.4.68-1.amzn2023.0.1.x86_64 13/13
Verifying : apr-1.7.5-1.amzn2023.0.4.x86_64 1/13
Verifying : apr-util-1.6.5-1.amzn2023.0.1.x86_64 2/13
Verifying : apr-util-lmdb-1.6.5-1.amzn2023.0.1.x86_64 3/13
Verifying : apr-util-openssl-1.6.5-1.amzn2023.0.1.x86_64 4/13
Verifying : generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch 5/13
Verifying : httpd-2.4.68-1.amzn2023.0.1.x86_64 6/13
Verifying : httpd-core-2.4.68-1.amzn2023.0.1.x86_64 7/13
Verifying : httpd filesystem-2.4.68-1.amzn2023.0.1.noarch 8/13
Verifying : httpd-tools-2.4.68-1.amzn2023.0.1.x86_64 9/13
Verifying : libbrotli-1.0.9-4.amzn2023.0.2.x86_64 10/13
Verifying : mailcap-2.1.49-3.amzn2023.0.3.noarch 11/13
Verifying : mod_httpd-2.0.42-1.amzn2023.0.1.x86_64 12/13
Verifying : mod_lua-2.4.68-1.amzn2023.0.1.x86_64 13/13

Installed:
apr-1.7.5-1.amzn2023.0.4.x86_64      apr-util-1.6.5-1.amzn2023.0.1.x86_64      apr-util-lmdb-1.6.5-1.amzn2023.0.1.x86_64
apr-util-openssl-1.6.5-1.amzn2023.0.1.x86_64      generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch      httpd-2.4.68-1.amzn2023.0.1.x86_64
httpd-core-2.4.68-1.amzn2023.0.1.x86_64      httpd filesystem-2.4.68-1.amzn2023.0.1.noarch      httpd-tools-2.4.68-1.amzn2023.0.1.x86_64
libbrotli-1.0.9-4.amzn2023.0.2.x86_64      mailcap-2.1.49-3.amzn2023.0.3.noarch      mod_httpd-2.0.42-1.amzn2023.0.1.x86_64
mod_lua-2.4.68-1.amzn2023.0.1.x86_64

Complete!
ec2-user@ip-172-31-0-53 ~$ sudo systemctl start httpd
ec2-user@ip-172-31-0-53 ~$ sudo systemctl enable httpd
rotated symlink /etc/systemd/system/multi-user.target.wants/httpd.service -> /usr/lib/systemd/system/httpd.service.
ec2-user@ip-172-31-0-53 ~$ sudo vi /var/www/html/index.html
ec2-user@ip-172-31-0-53 ~$ echo "OK - Web Server 1 is healthy" | sudo tee /var/www/html/health
OK - Web Server 1 is healthy
ec2-user@ip-172-31-0-53 ~$ curl http://localhost/health
OK - Web Server 1 is healthy
ec2-user@ip-172-31-0-53 ~$ []
```

```
aws
[Alt+S]
Installing : mod_httpd-2.0.42-1.amzn2023.0.1.x86_64 10/13
Installing : mod_lua-2.4.68-1.amzn2023.0.1.x86_64 11/13
Installing : generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch 12/13
Installing : httpd-2.4.68-1.amzn2023.0.1.x86_64 13/13
Running scriptlet: httpd-2.4.68-1.amzn2023.0.1.x86_64 13/13
Verifying : apr-1.7.5-1.amzn2023.0.4.x86_64 1/13
Verifying : apr-util-1.6.5-1.amzn2023.0.1.x86_64 2/13
Verifying : apr-util-lmdb-1.6.5-1.amzn2023.0.1.x86_64 3/13
Verifying : apr-util-openssl-1.6.5-1.amzn2023.0.1.x86_64 4/13
Verifying : generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch 5/13
Verifying : httpd-2.4.68-1.amzn2023.0.1.x86_64 6/13
Verifying : httpd-core-2.4.68-1.amzn2023.0.1.x86_64 7/13
Verifying : httpd filesystem-2.4.68-1.amzn2023.0.1.noarch 8/13
Verifying : httpd-tools-2.4.68-1.amzn2023.0.1.x86_64 9/13
Verifying : libbrotli-1.0.9-4.amzn2023.0.2.x86_64 10/13
Verifying : mailcap-2.1.49-3.amzn2023.0.3.noarch 11/13
Verifying : mod_httpd-2.0.42-1.amzn2023.0.1.x86_64 12/13
Verifying : mod_lua-2.4.68-1.amzn2023.0.1.x86_64 13/13

Installed:
apr-1.7.5-1.amzn2023.0.4.x86_64      apr-util-1.6.5-1.amzn2023.0.1.x86_64      apr-util-lmdb-1.6.5-1.amzn2023.0.1.x86_64
apr-util-openssl-1.6.5-1.amzn2023.0.1.x86_64      generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch      httpd-2.4.68-1.amzn2023.0.1.x86_64
httpd-core-2.4.68-1.amzn2023.0.1.x86_64      httpd filesystem-2.4.68-1.amzn2023.0.1.noarch      httpd-tools-2.4.68-1.amzn2023.0.1.x86_64
libbrotli-1.0.9-4.amzn2023.0.2.x86_64      mailcap-2.1.49-3.amzn2023.0.3.noarch      mod_httpd-2.0.42-1.amzn2023.0.1.x86_64
mod_lua-2.4.68-1.amzn2023.0.1.x86_64

Complete!
ec2-user@ip-172-31-8-147 ~$ sudo systemctl start httpd
ec2-user@ip-172-31-8-147 ~$ sudo systemctl enable httpd
rotated symlink /etc/systemd/system/multi-user.target.wants/httpd.service -> /usr/lib/systemd/system/httpd.service.
ec2-user@ip-172-31-8-147 ~$ sudo vi /var/www/html/index.html
ec2-user@ip-172-31-8-147 ~$ whoami
ec2-user
ec2-user@ip-172-31-8-147 ~$ echo "OK - Web Server 2 is healthy" | sudo tee /var/www/html/health
OK - Web Server 2 is healthy
ec2-user@ip-172-31-8-147 ~$ curl http://localhost/health
OK - Web Server 2 is healthy
ec2-user@ip-172-31-8-147 ~$ []
```

Task-4.Simulate a server failure by stopping one of your EC2 instances and observe how the ALB handles requests — describe what happens and how this demonstrates high availability.

Step 1: Stop Server-1

Go to: - **EC2 → Instances**

Select: - Server-1

Click: - **Instance state → Stop instance**

Confirm.

Wait for the instance to show:

Stopped

Step 2: Check the target group

Go to:- **EC2 → Target Groups → WebServers-TG → Targets**

After the health-check period, WebServer-1 should become:

Unhealthy

or otherwise no longer be eligible to receive traffic, depending on the exact state transition shown in the console.

Server-2 should remain:

Healthy

Step 3: Test the ALB

Open your ALB DNS name again:

http://YOUR-ALB-DNS-NAME

The website should continue working because the ALB detects that WebServer-1 is unavailable and sends requests to the remaining healthy WebServer-2.

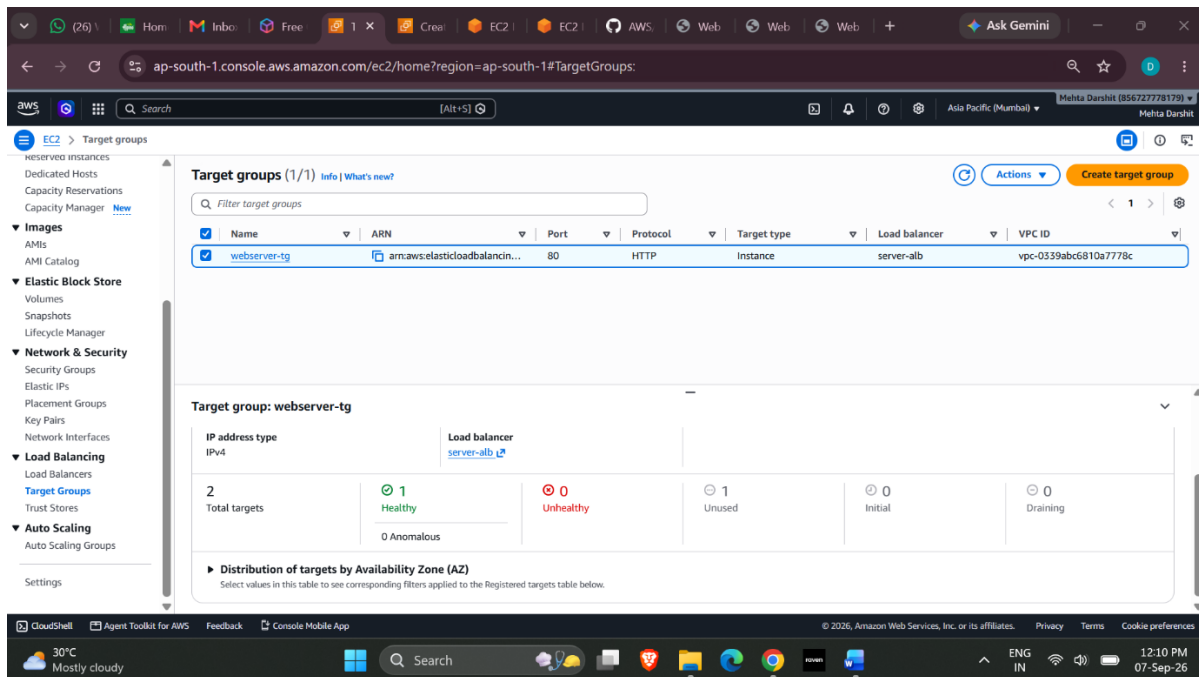
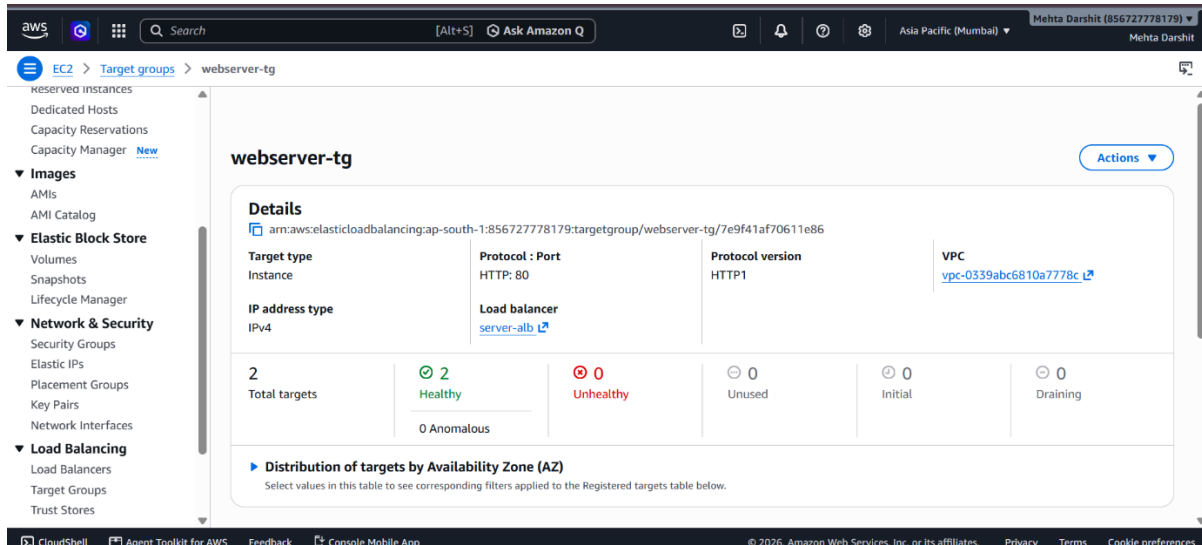
You should see:

Hello from Web Server 2

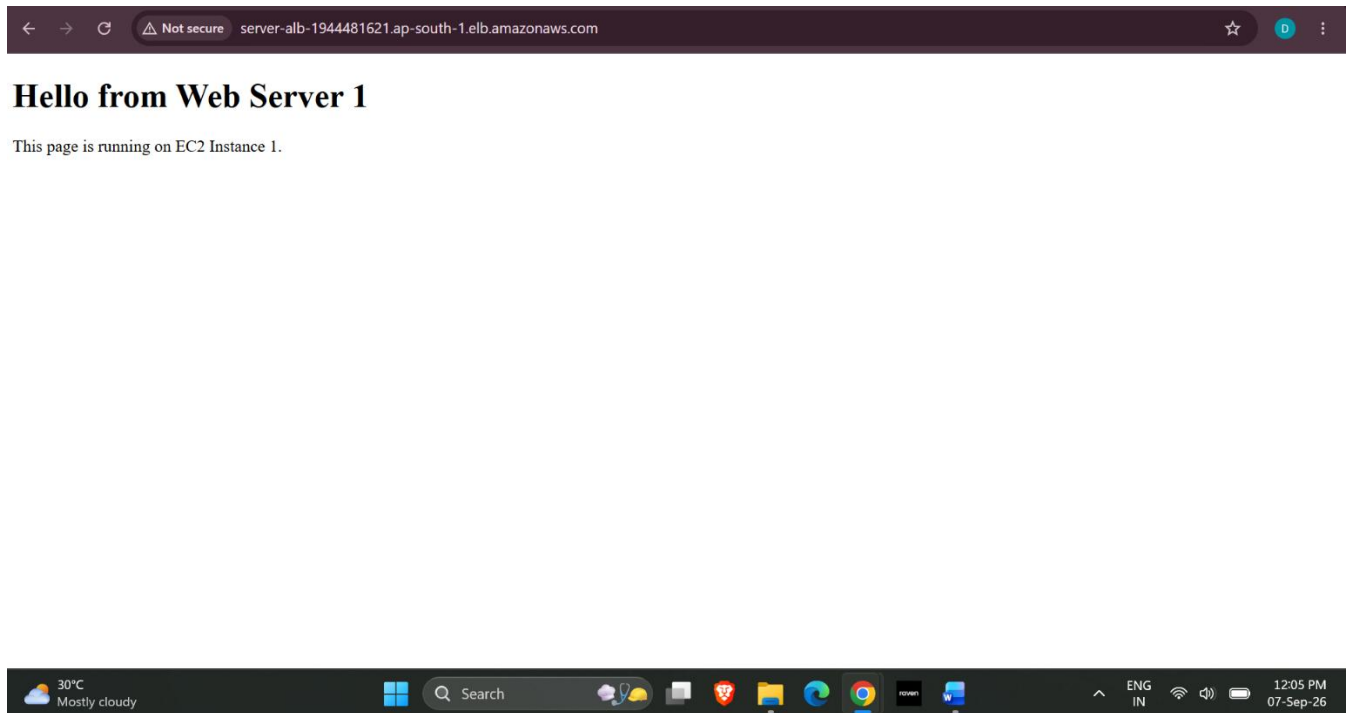
What this demonstrates

This demonstrates **high availability**.

If one server fails, the load balancer automatically stops sending traffic to that unhealthy server and directs users to the healthy server instead. Therefore, users can continue accessing the application without needing to know which backend server has failed.



After stop server 2 , I still access the web site



Task-5.Compare Application Load Balancer, Network Load Balancer, and Classic Load Balancer in a table with at least 3 points each, focusing on supported protocols, use cases, and features.

Feature	Application Load Balancer (ALB)	Network Load Balancer (NLB)	Classic Load Balancer (CLB)
Main protocol	HTTP, HTTPS, HTTP/2, WebSocket	TCP, TLS, UDP, QUIC	HTTP, HTTPS, TCP, TLS
OSI layer	Layer 7 (Application)	Layer 4 (Transport)	Layer 4/7 basic load balancing
Best use case	Websites, REST APIs, microservices, web applications	High-performance TCP/UDP applications and applications needing very low latency	Older applications using basic load-balancing requirements
Routing	Content-based routing using host/path rules	Connection/flow-based routing	Basic routing capabilities

Feature	Application Load Balancer (ALB)	Network Load Balancer (NLB)	Classic Load Balancer (CLB)
Performance	Excellent for HTTP/HTTPS applications	Very high performance and low latency	Basic/older capabilities
Advanced features	Path-based routing, host-based routing, redirects, authentication integrations, WebSockets	Static IPs, TLS termination, very high throughput, TCP/UDP support	Basic health checks and load balancing
Recommended today	Modern web applications	High-performance network applications	Mainly legacy workloads